

Implémentation, ServeurMultiThreade (v6)

Fabrice.Kordon@lip6.fr



Objectifs de cette version

2

Le service rend un résultat

- Facilité, résultat = requête envoyée
 - ▶ Permet de réutiliser la classe requête

Changements

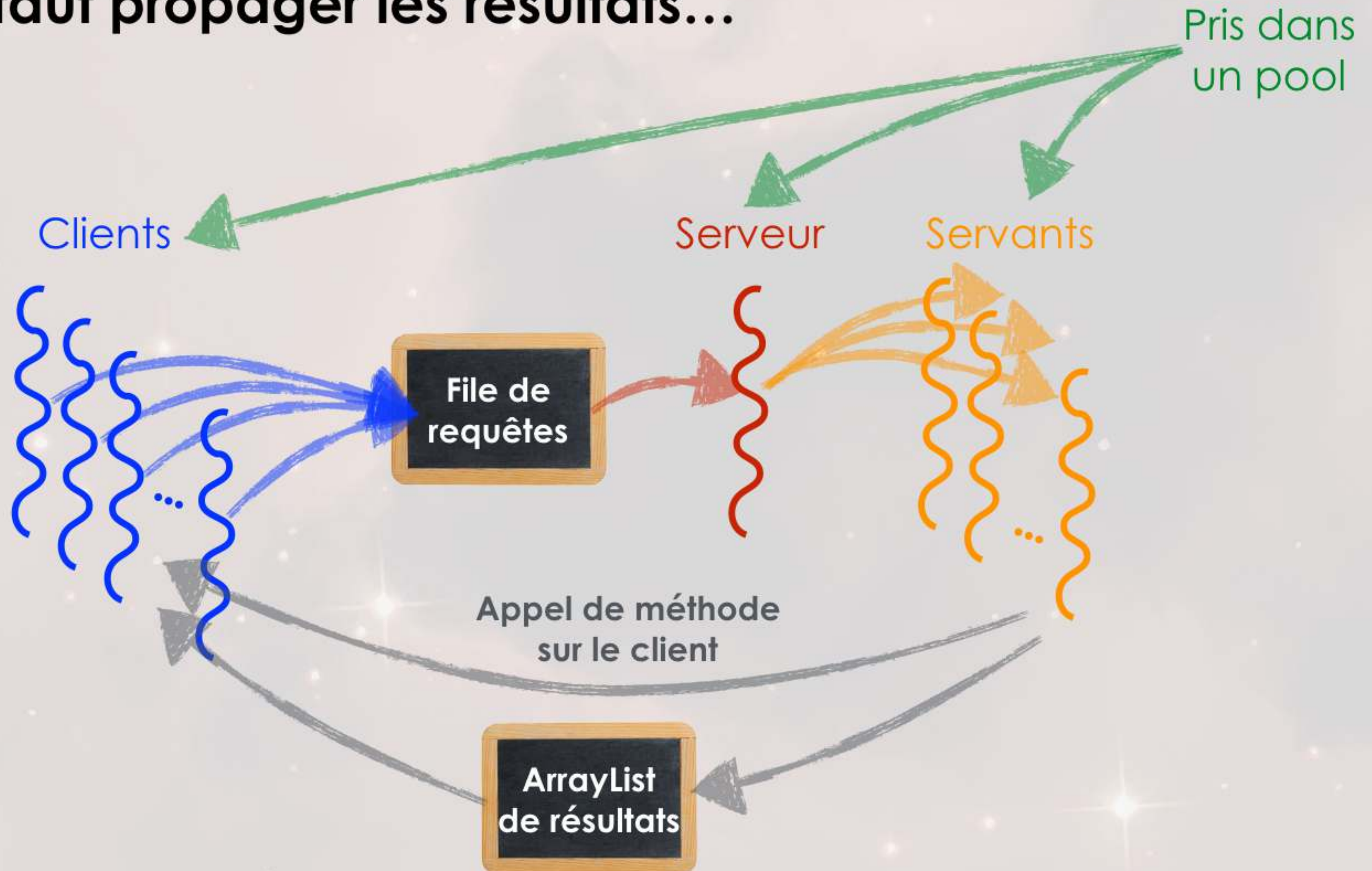
- ▶ Utilisation de Callable au bon endroit
- ▶ Besoin d'une file de réponses



Architecture modifiée

3

 Il faut propager les résultats...



Utilitaires + ce qui ne change pas

4

```
import java.util.Random;
import java.util.concurrent.ArrayBlockingQueue;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
import java.util.concurrent.ThreadFactory;
import java.lang.ThreadGroup;
import java.util.concurrent.TimeUnit;
import java.util.concurrent.Callable; // nouvelle dépendance
import java.util.concurrent.Future; // nouvelle dépendance
import java.util.ArrayList; // nouvelle dépendance

class MonUsineDeThread implements ThreadFactory {...}

class Requete {...}
```


La classe Servant

5

```
class Servant implements Callable<Requete> {
    private Requete maRequete;

    Servant (Requete r) {
        maRequete = r;
    }

    private void trace (String m) {
        System.out.println ("Servant " + maRequete + " : " + m);
        Thread.yield(); // rendre la commutation possible
    }

    public Requete call() { // Changement de signature
        this.trace("initialisé");
        try {
            Thread.sleep(maRequete.valeur);
        } catch (InterruptedException e) {
            System.out.println ("Servant, ne doit jamais arriver, " + e);
        }
        this.trace("je préviens le client");
        maRequete.emetteur.requeteTraitee();
        return maRequete; // renvoyer le résultat
    }
}
```


La classe LeServeur

6

```
class LeServeur implements Runnable {
    private ArrayBlockingQueue<Requete> f;
    // Borner le parallélisme des servants à 3
    private final int MAX_SERV = 3;
    private ExecutorService executor;
    ThreadGroup groupeServants = new ThreadGroup("servants");
    private ArrayList<Future<Requete>> res; // référence sur les résultats

    private ExecutorService poolClients; // les pools client

    LeServeur (ArrayBlockingQueue<Requete> fr,
               ExecutorService pc, ArrayList<Future<Requete>> r) {
        f = fr;
        res = r;
        poolClients = pc;
        MonUsineDeThread monUsine = new MonUsineDeThread(groupeServants);
        executor = Executors.newFixedThreadPool(MAX_SERV, monUsine);
    }

    private void trace (String m) {...}
}
```


La classe LeServeur

6

```
public void run() {
    Requete r = null;
    this.trace("initialisé");
    while (!poolClients.isTerminated()) {
        try {
            r = f.poll(1, TimeUnit.SECONDS); // Semi-passif nécessaire
            if (r != null) {
                this.trace("je traite " + r);
                Future<Requete> rq = executor.submit(new Servant(r));
                res.add(rq); // Ajout du «futur résultat»
            } else {
                this.trace("pool terminé = " + poolClients.isTerminated());
            }
        } catch (InterruptedException e) {
            System.err.println ("Serveur, ne doit jamais arriver, " + e);
            return;
        }
    }
    groupeServants.list();
    executor.shutdown();
    res.clear(); // Nettoyons ces résultats consommés
    this.trace("Adios");
}
}
```


La classe UnClient

7

```
class UnClient implements Runnable {
    private int monId;
    private ArrayBlockingQueue<Requete> f;

    private Random generateur = new Random();
    private final Object reveil = new Object(); // attente réponse

    private static int cpt = 1;
    private static final Object mutex = new Object();
    private ArrayList<Future<Requete>> res; // référence sur les résultats

    UnClient (ArrayBlockingQueue<Requete> fr, ArrayList<Future<Requete>> r) {
        f = fr;
        res = r;
        synchronized (mutex) {
            monId = cpt++;
        }
    }

    private void trace (String m) {...}

    public void requeteTraitee () {...}

    public String toString () {...}
}
```


La classe UnClient

7

```
public void run() {
    this.trace("initialisé");
    Requete r = new Requete(this, generateur.nextInt(1000));
    this.trace("j'envoie la requête " + r);
    try {
        f.put (r);
        this.trace("j'attends le serveur");
        synchronized (veille) {
            veille.wait();
        }
    } catch (InterruptedException e) {
        System.out.println ("UnClient "+monId+", cela doit jamais arriver, " + e);
    }
    this.trace("ma requête a été traitée");
    for (Future<Requete> futReq : res) { // parcourir une collection
        try{
            if (futReq.get().emetteur == this) {
                this.trace("sa valeur est " + futReq.get().valeur);
                //res.remove(futReq);
            }
        }
        catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```


La «classe principale»

8

```
public class ServeurMultiThreade6 {
    static int tailleFile;
    static int nbClients;

    public static void main(String []args) {
        try {
            tailleFile = Integer.parseInt(args[0]);
            nbClients = Integer.parseInt(args[1]);
        } catch (ArrayIndexOutOfBoundsException e) {
            System.err.println ("Arguments requis, taille de la file, nombre de clients.");
            return;
        }
        // Créer la file de requête et la collection de résultats
        ArrayBlockingQueue<Requete> laFile =
            new ArrayBlockingQueue<Requete>(tailleFile, true);
        ArrayList<Future<Requete>> lesResultats = new ArrayList<Future<Requete>>();

        // Créer les groupes du serveur et des clients
        ThreadGroup groupeServeur = new ThreadGroup("serveur");
        MonUsineDeThread usineServeur = new MonUsineDeThread(groupeServeur);
        ExecutorService execServeur = Executors.newSingleThreadExecutor(usineServeur);

        ThreadGroup groupeClients = new ThreadGroup("clients");
        MonUsineDeThread usineClients = new MonUsineDeThread(groupeClients);
        ExecutorService execClients = Executors.newFixedThreadPool(5, usineClients);
```


La «classe principale»

8

```
// Lancer le serveur dans son groupe
execServeur.execute(new LeServeur(laFile, execClients, lesResultats));
// Lancer les clients
for (int i = 0; i < nbClients ; i++) {
    execClients.execute(new UnClient(laFile, lesResultats));
}
// Terminer le système (quand tout aura été exécuté)
groupeServeur.list();
groupeClients.list();
execClients.shutdown();
execServeur.shutdown();
```

```
}
```

```
}
```


Une exécution

9

```
$ java ServeurMultiThreade6 2 3
Serveur : initialisé
UnClient 1 : initialisé
java.lang.ThreadGroup[name=serveur,maxpri=10]
UnClient 2 : initialisé
    UnClient 3 : initialisé
Thread[Thread-0,5,serveur]
java.lang.ThreadGroup[name=clients,maxpri=10]
    Thread[Thread-1,5,clients]
    Thread[Thread-2,5,clients]
    Thread[Thread-3,5,clients]
UnClient 2 : j'envoie la requête (client 2, 365)
UnClient 1 : j'envoie la requête (client 1, 618)
UnClient 3 : j'envoie la requête (client 3, 89)
UnClient 2 : j'attends le serveur
UnClient 3 : j'attends le serveur
UnClient 1 : j'attends le serveur
Serveur : je traite (client 2, 365)
Serveur : je traite (client 3, 89)
Servant (client 2, 365) : initialisé
Serveur : je traite (client 1, 618)
Servant (client 3, 89) : initialisé
Servant (client 1, 618) : initialisé
Servant (client 3, 89) : je prévient le client
UnClient 3 : ma requête a été traitée
Servant (client 2, 365) : je prévient le client
UnClient 2 : ma requête a été traitée
UnClient 2 : sa valeur est 365
UnClient 3 : sa valeur est 89
Servant (client 1, 618) : je prévient le client
UnClient 1 : ma requête a été traitée
```

```
UnClient 1 : sa valeur est 618
Serveur : pool terminé = true
java.lang.ThreadGroup[name=servants,maxpri=10]
    Thread[Thread-4,5,servants]
    Thread[Thread-5,5,servants]
    Thread[Thread-6,5,servants]
Serveur : Adios
```


Une exécution

9

```
$ java ServeurMultiThreade6 2 3
Serveur : initialisé
UnClient 1 : initialisé
java.lang.ThreadGroup[name=serveur,maxpri=10]
UnClient 2 : initialisé
  UnClient 3 : initialisé
Thread[Thread-0,5,serveur]
java.lang.ThreadGroup[name=clients,maxpri=10]
  Thread[Thread-1,5,clients]
  Thread[Thread-2,5,clients]
  Thread[Thread-3,5,clients]
UnClient 2 : j'envoie la requête (client 2,
UnClient 1 : j'envoie la requête (client 1,
UnClient 3 : j'envoie la requête (client 3,
UnClient 2 : j'attends le serveur
UnClient 3 : j'attends le serveur
UnClient 1 : j'attends le serveur
Serveur : je traite (client 2, 365)
Serveur : je traite (client 3, 89)
Servant (client 2, 365) : initialisé
Serveur : je traite (client 1, 618)
Servant (client 3, 89) : initialisé
Servant (client 1, 618) : initialisé
Servant (client 3, 89) : je prévient le client
UnClient 3 : ma requête a été traitée
Servant (client 2, 365) : je prévient le client
UnClient 2 : ma requête a été traitée
UnClient 2 : sa valeur est 365
UnClient 3 : sa valeur est 89
Servant (client 1, 618) : je prévient le client
UnClient 1 : ma requête a été traitée
```

```
UnClient 1 : sa valeur est 618
Serveur : pool terminé = true
java.lang.ThreadGroup[name=servants,maxpri=10]
  Thread[Thread-4,5,servants]
  Thread[Thread-5,5,servants]
  Thread[Thread-6,5,servants]
Serveur : Adios
```



Complément à la v5?

Affichages du résultat des requêtes

En guise de conclusion...

10

On gère le retour d'un résultat

- Même si artificiel (mécanique présente)

Mais...

- Le «nettoyage» ne peut-il pas être «mieux»?

